

Describer-Decider Engine v33.0 — Complete System Explanation

Table of Contents

- 1. [System Overview](#)
- 2. [Architecture & Data Flow](#)
- 3. [File-by-File Breakdown](#)
 - [3.1 runner.py — Colab Entry Point](#)
 - [3.2 models.py — Data Models & Core Helpers](#)
 - [3.3 input_adapter.py — Format Detection & Normalization](#)
 - [3.4 agents.py — GPT-4o API Agents](#)
 - [3.5 graph.py — LangGraph Orchestration](#)
 - [3.6 parser/log_parser.py — Unified Log Ingestion](#)
- 4. [LangGraph Node Reference](#)
- 5. [Pydantic Model Reference](#)
- 6. [Prompt Engineering Deep-Dive](#)
- 7. [Cascading Failure Analysis](#)
- 8. [Log Handling System](#)
- 9. [Input Format Support](#)
- 10. [Error Handling & Resilience](#)
- 11. [Performance & Configuration](#)

1. System Overview

The Describer-Decider Engine is an **AI-powered fault localization pipeline** for UI-driven applications. Given a recorded execution trace of a mobile or web application (screenshots before/after each step, an action log, and optionally runtime logs), the system determines which steps failed, classifies the failure type, links failures into causal chains, and produces a structured diagnostic report.

Core Hypothesis

A Vision-Language Model (VLM) can observe UI state changes between a "before" and "after" screenshot, and a reasoning LLM can then judge whether the application behaved correctly — producing machine-readable fault reports.

Key Design Choices

Decision	Rationale
Two-agent separation (Observer + Decider)	Keeps observation objective; only the Decider applies pass/fail rules
GPT-4o for all agents	API-based, no GPU dependency, high OCR accuracy

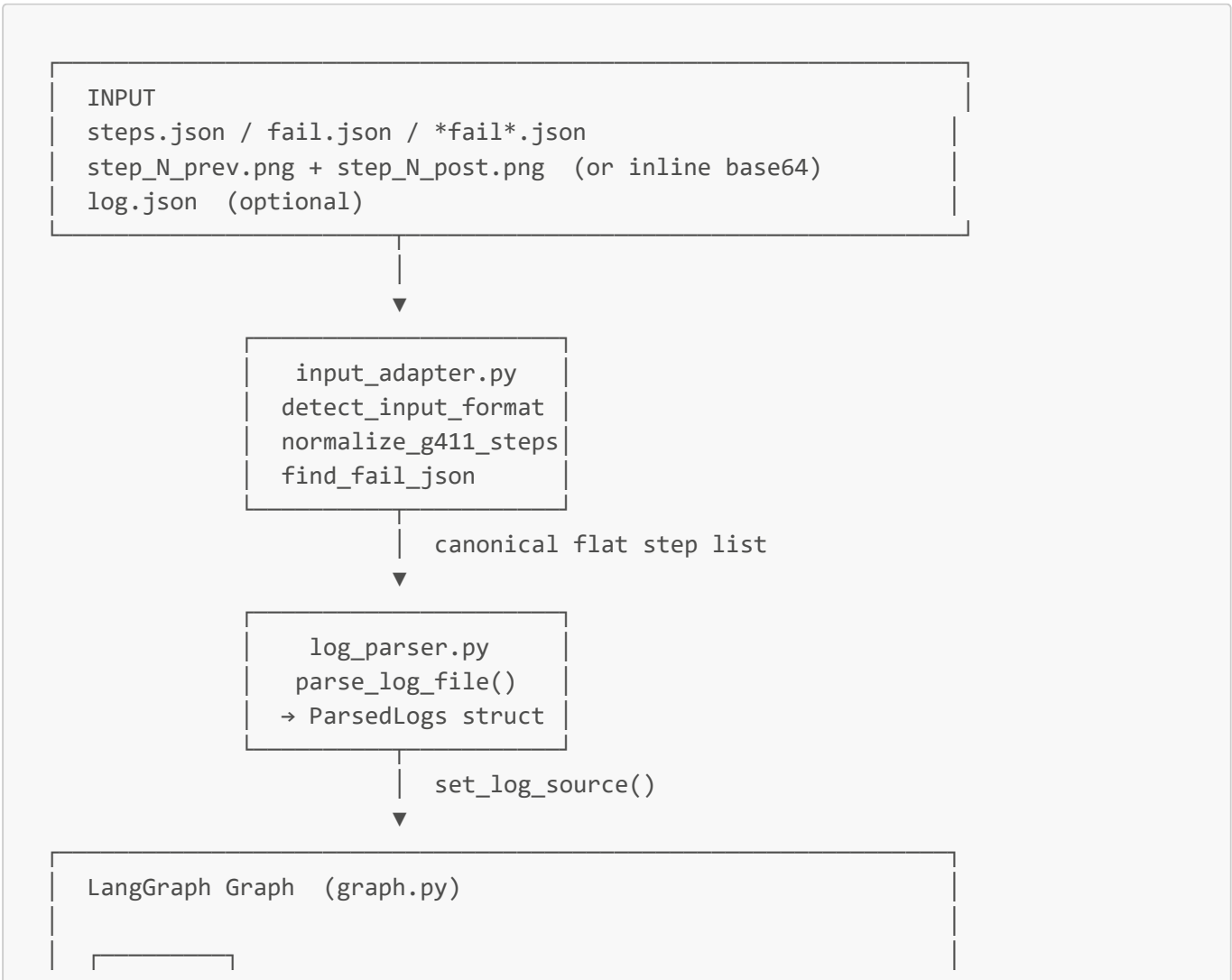
Decision	Rationale
LangGraph	Deterministic, auditable sequential graph — required for reproducible academic research
Pydantic v2	Strict typed I/O for every agent boundary; prevents silent data corruption
Causal chain analysis	Groups failures rather than listing them flat; shows root vs. downstream symptoms

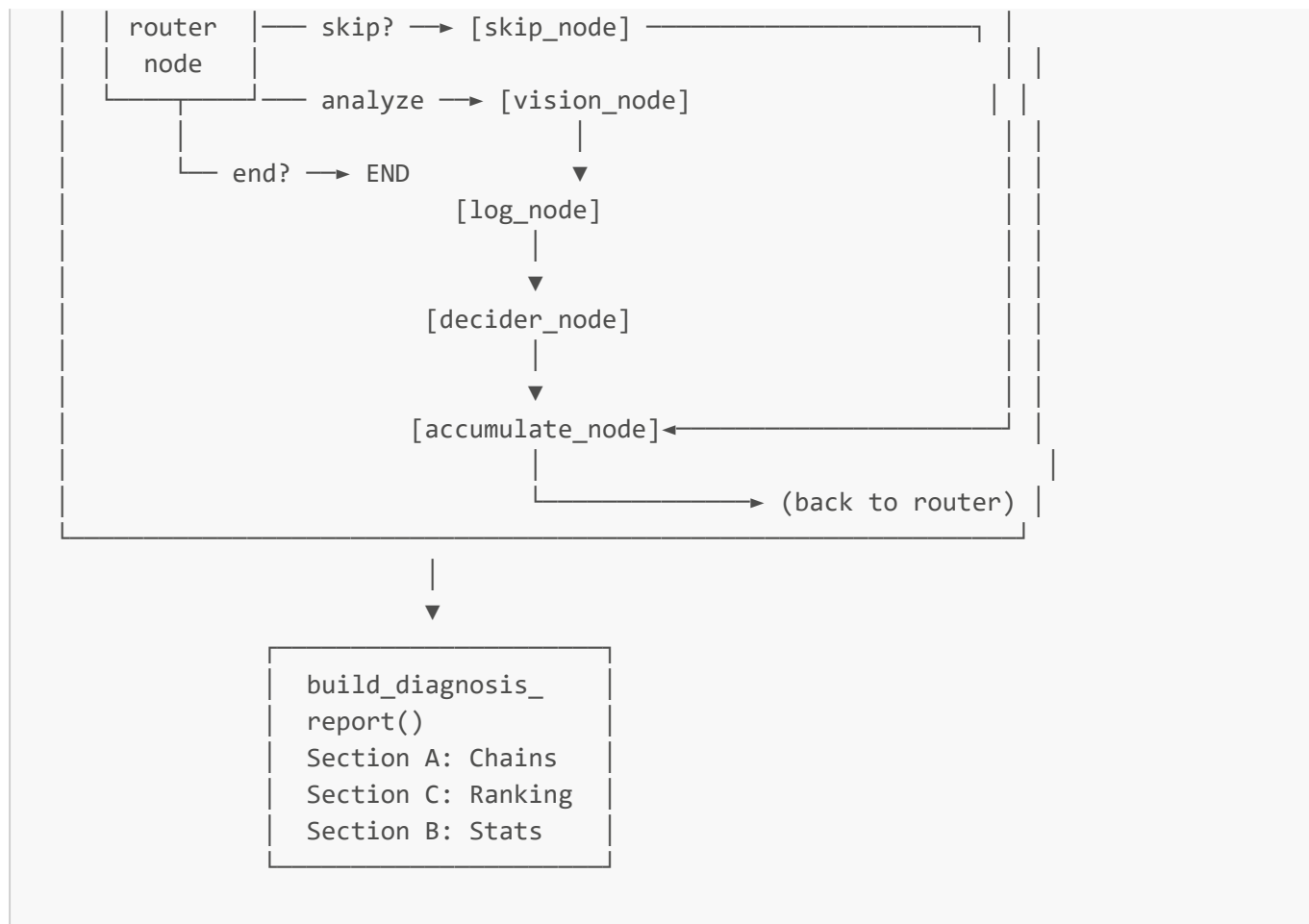
Validated Performance (v33.0)

Evaluated on **12 apps / 136 steps**:

Dataset	Steps	Precision	Recall	F1
Our Traces (7 apps)	85	100.0%	100.0%	100.0%
Group 411 (5 cases)	46	91.7%	84.6%	88.0%
Combined	131	95.8%	92.0%	93.9%

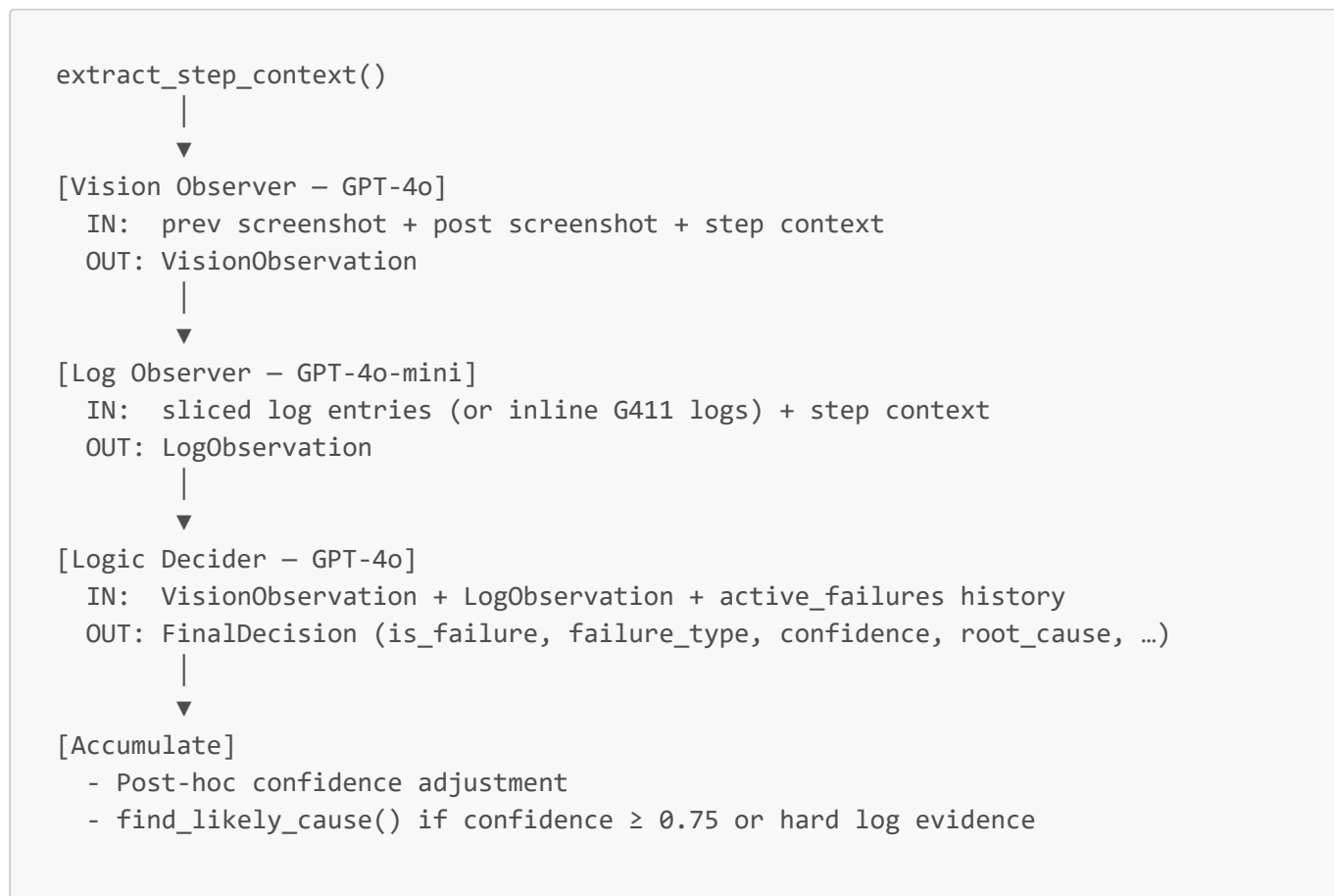
2. Architecture & Data Flow





Per-Step Processing Pipeline

For every non-skipped step, the graph runs these stages in sequence:



- Append to `step_results`, `detected_errors`, `active_failures`
- Advance step counter → back to router

3. File-by-File Breakdown

3.1 `runner.py` — Colab Entry Point

Purpose: Bootstrap the entire pipeline in a Google Colab notebook. It handles environment setup, authentication, data loading, log parsing, and kicks off the graph.

What It Does Step by Step

1. Dependency Installation

```
!pip install -q openai langchain langchain-core "langgraph>=0.2,<0.3" pydantic nest_asyncio
```

The `!` prefix runs a shell command inside Colab. All deps are pinned to avoid breaking changes.

`nest_asyncio` is needed because Colab already has a running event loop and LangGraph's async internals would otherwise raise a `RuntimeError`.

2. Path Setup

```
PROJECT_PATH = "/content/drive/MyDrive/Grad Project"
STEPS_PATH = f"{PROJECT_PATH}/steps"
for _p in (PROJECT_PATH, os.path.join(PROJECT_PATH, "parser")):
    sys.path.insert(0, _p)
```

Both the project root and the `parser/` subdirectory are inserted into `sys.path`. This allows `import models`, `import agents`, `import graph`, and `from log_parser import parse_log_file` to all work without package installation.

3. API Key Authentication

```
if "OPENAI_API_KEY" not in os.environ:
    os.environ["OPENAI_API_KEY"] = input("OpenAI API Key: ")
```

If the key is already in the environment (e.g., set via Colab Secrets), the `input()` prompt is skipped.

4. Google Drive Mount

```
from google.colab import drive
if not os.path.exists('/content/drive'):
    drive.mount('/content/drive')
```

All input data (screenshots, JSON files) lives on Google Drive. The mount check avoids remounting if already done.

5. Data Loading

```
steps_data = load_project_data(STEPS_PATH)
```

Delegates to `models.load_project_data()`, which internally calls `input_adapter` to detect format and normalize. Returns a flat list of step dicts.

6. Log Parsing

```
_parsed_logs = parse_log_file(os.path.join(STEPS_PATH, "log.json"),
total_steps=len(steps_data))
set_log_source(_parsed_logs)
```

`parse_log_file` auto-detects the log format. `set_log_source` installs the result into a module-level variable in `graph.py` so all `log_nodes` can access it without passing it through LangGraph state.

7. Graph Invocation

```
res = app.invoke({
    "current_step_index": 0,
    "steps_data": steps_data,
    "trace_files_path": STEPS_PATH,
    ...
}, config={"recursion_limit": 150})
```

The recursion limit of 150 covers datasets up to ~70 analyzed steps (each step loops `router→vision→log→decider→accumulate→router` = ~5 hops). The initial `GraphState` is fully specified here — all transient fields (`_current_ctx`, `_current_vision_obs`, etc.) start as `None`.

3.2 `models.py` — Data Models & Core Helpers

Purpose: Defines all Pydantic models, the LangGraph `TypedDict` state, and every stateless helper function used by the graph nodes.

Pydantic Models

VisionObservation — Output of the Vision Observer agent.

```
prev_screen_elements:  What is visible on the PREV screenshot (element-level)
post_screen_changes:   What changed on the POST screenshot (element-level)
semantic_screen_summary: 1-2 sentence high-level screen description after the
action
```

FinalDecision — Output of the Logic Decider agent.

```
expected_value:      Specific data value the step should confirm (or "N/A")
observed_value:      Actual data value seen in POST (or "N/A" / "NOT CONFIRMED")
expected_behavior:   What SHOULD happen for this step (derived from instruction)
observed_behavior:   What ACTUALLY happened (derived from observer report)
is_failure:         Boolean verdict
failure_type:       "element_missing" | "action_failed" | "content_mismatch" |
"system_error" | "none"
observation:        One-line verdict summary
root_cause:         Root cause string (empty on pass)
confidence:         Float 0.0-1.0, self-assessed by Logic Decider
```

LogEntry — A single parsed log event.

```
timestamp: str
level:     Literal["info", "warning", "error", "critical"]
source:    Literal["network", "console", "system", "application"]
message:   str
```

NetworkEvent — A single HTTP request/response.

```
method, url, status_code, latency_ms, error_message
```

LogObservation — Output of the Log Observer agent.

```
has_logs:           Were any logs found?
log_summary:        1-3 sentence plain-English summary
relevant_errors:    Up to 5 LogEntry objects
network_events:     Up to 5 NetworkEvent objects
has_network_failure: Any 4xx/5xx or timeout?
has_system_error:   Any error/critical level entry?
latency_anomaly:    Any request > 5000ms?
```

GraphState (TypedDict) — The full mutable state passed through LangGraph nodes.

Persistent across all steps:

```

current_step_index: Which step we're processing next
steps_data: Full list of normalized step dicts
trace_files_path: Path to the folder containing images and logs
final_report: Assembled at the end
stop_execution: Signal to terminate the graph
detected_errors: List of failure records
active_failures: Rolling list of failure summary strings (injected into
Decider)
active_failure_map: Dict[str(step_idx) → failure dict] for causal linking
step_results: Per-step record for ALL analyzed steps (pass + fail)

```

Transient per-step (overwritten each iteration):

```

_current_ctx: Step context extracted by router_node
_current_vision_obs: Vision agent output (as dict)
_current_log_obs: Log agent output (as dict)
_current_decision: Decider output (as dict)
_route: "skip" | "analyze"
_skip_reason: Why the step was skipped

```

Helper Functions

load_project_data(folder_path) Locates the JSON trace file via **find_fail_json**, auto-detects format, normalizes if G411, and returns a flat list. Tries three encodings (utf-8, utf-8-sig, cp1254) to handle Turkish text files.

extract_step_context(raw_item, current_idx) Normalizes a raw step dict (flat v33 or nested G411) into a canonical context dict with **step_num**, **action**, **instruction**, **target**, **requires_vlm**. Critical logic: for **waitUntil** actions, drills into the nested **condition** sub-object to extract the real **element** and **instruction**, since they live one level deeper in these steps.

robust_json_extract(text) Two-pass JSON extractor. First tries to find a fenced ````json```` block, then falls back to a greedy `{...}` regex match. Replaces Python-style **True/False/None** with JSON equivalents before parsing.

format_active_failures(failures) Simple formatter: joins the rolling **active_failures** list with newlines, or returns a "healthy" sentinel string.

_normalize_keywords(text) Tokenizer for log-to-step matching. Strips Turkish diacritics (ç→c, ğ→g, ı→i, ö→o, ş→s, ü→u). Splits on whitespace, arrows, commas, brackets, etc., but **preserves dots and colons surrounded by digits** so that time formats like **12:15** or **13.35** survive as single tokens. Returns a lowercase set of tokens with 3+ characters.

extract_step_logs(ctx, all_logs, step_idx, total_steps) Two-pass log slicer for dynamic (raw unstepped) log files:

- **Pass 1 (Keyword):** Scores each log entry against the step's `target` and `instruction` keywords. Entries matching the `element_id` field get double weight. Returns up to 20 top-scoring entries if at least 3 keyword matches are found.
- **Pass 2 (Positional fallback):** If fewer than 3 keyword matches, falls back to a window of log entries at the proportional position in the log file (e.g., step 3 of 10 = roughly entries 30%–40% of the way through the file).
- `vlm_agent` source entries are excluded from both passes to prevent test-framework assertion records from contaminating the keyword scoring.

3.3 `input_adapter.py` — Format Detection & Normalization

Purpose: Makes the pipeline format-agnostic. Handles two incompatible trace formats (v33 and Group 411) and translates both into the same canonical dict structure.

Format Detection

`detect_input_format(data)`

G411 fingerprint (all three must match):

1. Top-level dict with a `stepDetails` list
2. Each item has a nested `step` sub-object with only `{action, stepInstruction, condition}` keys
3. Each item carries `prevScreenshot` + `currScreenshot` base64 fields AND has no `step_idx`

If all three match → `"g411"`. Otherwise → `"v33"`.

G411 Normalization

`normalize_g411_steps(data)`

Converts each G411 `stepDetails` entry into the canonical flat format:

```
{
  "step_idx":      idx,          # assigned by array position (G411 has no
step_idx)
  "action":        mapped_action, # "verify"→"abstractVerification",
"type"→"click"
  "stepInstruction": ...,
  "element":       instruction,   # G411 has no element field; use instruction as
proxy
  "requiresVLM":   True,
  "_inline_prev_b64": ...,        # raw base64 (no data: prefix)
  "_inline_curr_b64": ...,
  "_inline_console_logs": ...,    # plain text, may be empty
  "_inline_network_logs": ...,
}
```

The `_inline_*` private keys allow `vision_node` and `log_node` to detect G411 inputs and deliver data without any disk I/O. The `data:` URI prefix is added only at the moment the OpenAI API call is constructed (in

`agents.py`).

Action mapping:

```
"verify" → "abstractVerification" (same semantics: confirm a UI state)
"type"   → "click"                 (treated as click; vision confirms text
appeared)
```

File Discovery

`find_fail_json(folder_path)`

Search order:

1. `steps.json` (v33 primary convention)
2. `fail.json` (v33 alternate)
3. `glob *fail*.json` (G411 naming: `passo_fail.json`, `bank-app-fail.json`, etc.)

Returns the first match, or `None` if nothing is found.

MIME Detection

`detect_mime_from_b64(b64)`

Infers image type from the first few base64 characters:

- Starts with `/9j` → JPEG (base64 of JPEG magic bytes `FF D8`)
- Anything else → PNG (base64 of `\x89PNG`)

Used by `agents.py` when constructing the `image_url` content block for G411 inline images.

3.4 `agents.py` — GPT-4o API Agents

Purpose: Three GPT-4o agents (Vision Observer, Logic Decider, Log Observer) exposed as LangChain `RunnableLambda` wrappers. The graph calls these via `vision_chain.invoke()`, `logic_chain.invoke()`, and `log_chain.invoke()`.

API Client

`_get_client(force_new=False)` Lazy-initializes a single `OpenAI` client. Pass `force_new=True` to discard the existing client and create a fresh one — used during retry after a socket-level error.

`_api_call_with_retry(call_fn, max_retries=3)` Wraps every API call with retry logic:

- On **attempt 0**: call directly.
- On **attempt 1+**: re-create client (`force_new=True`), sleep 2^{attempt} seconds (2s, 4s), retry.
- Retries on: `OSError` with `errno` 107 (ENOTCONN) or 111 (ECONNREFUSED) (Colab socket issues), or API errors matching rate limit / 502 / 503 / timeout / connection keywords.
- Non-retriable errors (e.g. auth failure, invalid request) propagate immediately.

Vision Observer

run_vision_observer(inputs)

Inputs:

- `ctx`: step context dict
- `prev/post`: disk paths (v33 mode), OR
- `prev_b64/post_b64`: raw base64 strings (G411 inline mode)

Model: `gpt-4o`, `temperature=0`, `max_tokens=500`, `response_format={"type":"json_object"}`

The agent receives **two images** plus a detailed text prompt, in a single user message:

```
{
  "role": "user",
  "content": [
    {
      "type": "image_url",
      "image_url": {
        "url": "data:image/png;base64,..."
      },
      "detail": "high"
    },
    {
      "type": "image_url",
      "image_url": {
        "url": "data:image/png;base64,..."
      },
      "detail": "high"
    },
    {
      "type": "text",
      "text": prompt
    }
  ]
}
```

`detail="high"` enables OpenAI's high-resolution image tiling, critical for accurate OCR of small UI elements.

The prompt defines three tasks:

1. **TASK 1 — TARGET SEARCH:** Is the target element in PREV/POST? Handles: state changes, semantic equivalence, selected state, overlay detection.
2. **TASK 2 — VISUAL CHANGES:** Screen context changes, new content sections, strict value OCR (only for `abstractVerification` with a specific data value).
3. **TASK 3 — SEMANTIC SCREEN SUMMARY:** 1-2 sentence overall description.

Output is parsed with `robust_json_extract` and validated into a `VisionObservation`.

Logic Decider

run_logic_decider(inputs)

Inputs:

- `ctx`, `vision_report` (`VisionObservation`), `log_report` (`LogObservation`), `active_failures` (list)

Model: `gpt-4o`, `temperature=0`, `max_tokens=450`, `response_format={"type":"json_object"}`

Two-phase design:

- **Phase 1 — Verdict:** Apply Decision Rules 0–3 in strict priority order using only visual + log evidence.
- **Phase 2 — Root Cause:** After committing to a verdict, consult the last 3 `active_failures` entries to enrich `root_cause`. The failure history is placed **after** the rules in the prompt to prevent anchoring bias on the `is_failure` verdict.

Log injection: When `log_report.has_logs=True`, a `[LOG EVIDENCE]` block and `[LOG EVIDENCE RULES]` block are appended to the prompt. When `has_logs=False`, these strings are empty, making the prompt byte-for-byte identical to the log-less version.

Decision Rules (applied in strict priority order):

Rule	Name	Condition → Outcome
0	System Overlay Exception	System dialog over main screen → SUCCESS
0B	App Loading Exception	click + loading spinner appeared in POST → SUCCESS
0C	Target State Change Success	click + Observer reports state/content change or selected state → SUCCESS
1	Target Absent	Target absent from both PREV and POST → <code>element_missing</code> (with exceptions for close/dismiss, navigation, wrong-item, wrong-value navigation)
2	Strict Value Verification	<code>abstractVerification</code> or explicit data value verify: Observer must confirm exact value → <code>content_mismatch</code> if not confirmed
3	Success	No failure condition matched

Output is parsed into a `FinalDecision`. On parse failure, a fallback `FinalDecision(is_failure=True, failure_type="system_error")` is returned rather than crashing.

Log Observer

`run_log_observer(inputs)`

Inputs (two modes):

- `log_text`: pre-sliced log string (unified mode)
- `log_file_path`: path to per-step file (legacy mode)

If neither exists or both are empty, returns `LogObservation(has_logs=False)` immediately without an API call.

Model: `gpt-4o-mini`, `temperature=0`, `max_tokens=600`

Pre-processing: Log text is normalized before sending: uppercase `"INFO"` → `"info"`, `"ERROR"` → `"error"`, `"WARNING"` → `"warning"`, `"DEBUG"` → `"info"`. This ensures Pydantic's `Literal` field validation doesn't reject entries.

The prompt asks the model to extract a summary, relevant errors, and network events, and set three boolean flags (`has_network_failure`, `has_system_error`, `latency_anomaly`).

`_normalize_log_obs_dict(parsed)` is applied after JSON parsing and before Pydantic validation to coerce:

- `level` → lowercase, must be in `{"info", "warning", "error", "critical"}`

- `source` → lowercase, must be in `{"network", "console", "system", "application"}`
- `latency_ms` → float (strips "ms" suffix if present)
- `status_code` → int

LangChain Wrappers

```
vision_chain = RunnableLambda(run_vision_observer)
logic_chain  = RunnableLambda(run_logic_decider)
log_chain    = RunnableLambda(run_log_observer)
```

These are the objects imported by `graph.py`. The `RunnableLambda` wrapper gives each agent the full LangChain `.invoke()` interface.

3.5 `graph.py` — LangGraph Orchestration

Purpose: Defines the 6-node LangGraph `StateGraph`, wires the edges, manages the unified log store, and provides image-path resolution utilities.

Module-Level Log Store

```
_parsed_logs: ParsedLogs | None = None

def set_log_source(parsed: ParsedLogs) -> None: ...
def set_unified_logs(logs: list) -> None: ... # legacy shim
```

The `ParsedLogs` struct is kept as a module-level variable (not in `GraphState`) because LangGraph serializes its state as JSON, and a `ParsedLogs` object with thousands of log entries would be serialized and deserialized on every node transition — extremely wasteful. Module-level storage means all nodes share one reference with zero serialization overhead.

`set_log_source()` is the canonical API. `set_unified_logs()` is a backward-compatible shim that wraps a raw list into a dynamic-mode `ParsedLogs`.

Image Helpers

`find_image(folder, step_num, suffix)` Tries `.png`, `.jpg`, `.jpeg` extensions in that order. Returns the first path that exists, or `None`.

`_get_image_inputs(step_item, folder, step_num)` Returns the correct image payload for the vision agent:

- If the step dict has `_inline_prev_b64` and `_inline_curr_b64` (G411 mode) → returns `{"prev_b64": ..., "post_b64": ...}`
- Otherwise → calls `find_image` for both prev/post and returns `{"prev": path, "post": path}`

Node 1: router_node

The decision-making hub. Called at the start of every loop iteration.

End condition: `current_step_index >= len(steps_data)` → assembles the final report via `build_diagnosis_report()`, sets `stop_execution=True`.

Context extraction: `extract_step_context(steps[idx], idx)` normalizes the raw step dict.

Skip conditions:

- `action == "wait"` — time delays don't need visual analysis
- `requires_vlm == False` — steps explicitly flagged to skip vision
- `target == "null"` — no element to analyze
- Images missing from disk AND no inline base64

Sets: `_current_ctx, _route ("skip" or "analyze"), _skip_reason`

`route_step(state)` — conditional edge function: returns "end" if `stop_execution`, else the `_route` value.

Node 2: skip_node

Logs the skip reason and increments `current_step_index` by 1. No API calls made. Steps skipped here are not included in `step_results` and are counted as "skipped" in the final summary.

Node 3: vision_node

Resolves image inputs via `_get_image_inputs`, invokes `vision_chain`. On any exception, returns a fallback `VisionObservation` with "System error" strings rather than crashing the graph. Stores the result as `_current_vision_obs` (a dict, not the Pydantic object — LangGraph state must be JSON-serializable).

Node 4: log_node

Priority order for log data:

1. **G411 inline logs** (highest priority): if `_inline_console_logs` or `_inline_network_logs` are present in the step dict, concatenate and send directly to `log_chain`. No external log source checked.
2. **Preassigned mode:** if `_parsed_logs.mode == "preassigned"`, look up `step_map[step_num]`. Returns exactly the entries pre-assigned to this step by the log parser.
3. **Dynamic mode:** if `_parsed_logs.mode == "dynamic"`, call `extract_step_logs(ctx, raw_entries, step_num, total_steps)` for keyword+positional assignment.
4. **Legacy mode** (fallback): if `_parsed_logs` is `None` or mode is "empty", try reading `step_{step_num}_logs.json` from the trace folder.

Stores result as `_current_log_obs` (dict).

Node 5: decider_node

Reconstructs `VisionObservation` and `LogObservation` Pydantic objects from the stored dicts. Invokes `logic_chain` with the full evidence package. On exception, returns a `FinalDecision(is_failure=True, failure_type="system_error")` fallback.

Stores result as `_current_decision` (dict).

Node 6: `accumulate_node`

The bookkeeping hub. Does no API calls — only updates state.

Post-hoc confidence adjustment:

```
If log_summary starts with "Log parsing" → confidence -= 0.10 (log data
corrupt)
If has_logs == False → confidence -= 0.05 (flying blind)
Floor at 0.1
```

The Logic Decider cannot know whether log parsing failed — this correction is applied externally after the decision.

Causal chain gating: `find_likely_cause()` is called **only when** `confidence >= 0.75` OR hard log evidence exists (`has_network_failure` or `has_system_error`). This prevents low-confidence false positives (e.g., mid-animation misreads) from anchoring spurious chains to earlier steps.

State updates:

- Appends a record to `step_results` for every step (pass or fail)
- On failure: appends to `detected_errors`, `active_failures`, `active_failure_map`
- Increments `current_step_index`

Graph Wiring

```
workflow.set_entry_point("router")
workflow.add_conditional_edges("router", route_step, {"skip": "skip", "analyze":
"vision", "end": END})
workflow.add_edge("vision", "log")
workflow.add_edge("log", "decider")
workflow.add_edge("decider", "accumulate")
workflow.add_edge("skip", "router")
workflow.add_edge("accumulate", "router")
```

The `skip` and `accumulate` nodes both loop back to `router`, making this a cyclic graph that processes one step per cycle.

3.6 `parser/log_parser.py` — Unified Log Ingestion

Purpose: Auto-detects the format of a `log.json` file and returns a `ParsedLogs` struct that the graph can consume uniformly across all four supported log shapes.

ParsedLogs Dataclass

```
@dataclass
class ParsedLogs:
    mode: Literal["preassigned", "dynamic", "empty"]
    format_name: str
    step_map: Dict[int, List[dict]] # populated when mode == "preassigned"
    raw_entries: List[dict] # populated when mode == "dynamic"
    warnings: List[str] # surfaced by runner.py at startup
```

Three modes:

- **preassigned:** step ownership is known at parse time — `step_map[step_idx]` gives the exact entries for that step.
- **dynamic:** raw unstepped entries — the graph calls `extract_step_logs()` at query time.
- **empty:** no file, empty file, or unparseable — all log lookups return empty lists.

parse_log_file(log_path, total_steps)

Entry point. Checks file existence, loads JSON, dispatches to the appropriate sub-parser based on `_detect_format(raw)`.

Format Detection — _detect_format(raw)

Inspects only the **first entry** of the array:

- `first["step"]` is a `dict` → `nested`
- `first["step"]` is an `int` → `flat_indexed`
- Scans first 20 entries for `source=="StepRunner"` + assertion regex → `flat_assertions`
- Otherwise → `raw`

Format A1 — nested (_parse_step_nested)

Array where **array index = step_idx**. Each item is `{step: {...}, status, logs?: str}`.

The `logs` field is a pipe-delimited string: `key: value | key: value | ...`, with multiple rows joined by `\n`. `_parse_pipe_delimited` splits on `|` boundaries, then on the first `:` in each field, producing a list of dicts.

Missing `logs` fields are recorded as empty buckets with a warning counter.

Format A2 — flat_indexed (_parse_flat_indexed)

Flat array where each entry has `step: <int>`.

1-indexed shift detection: If `min(step_values) == 1` and `0 not in step_values`, the log is 1-indexed (step 1 corresponds to `step_idx=0`). All keys are shifted down by 1 with a warning. The old check required

`max_v == total_steps` too, but this was removed because trailing skipped steps (e.g., a final `wait` action) make `max_v < total_steps` even for genuine 1-indexed logs.

Out-of-range entries (after any shift) are flagged in warnings.

Format B — `flat_assertions` (`_parse_flat_assertions`)

Flat array with **no `step` field** on individual entries. Step boundaries are inferred from `StepRunner` log entries whose `message` matches `"Step N passed/failed"`.

Algorithm: accumulate entries into a rolling `bucket`. When a `StepRunner` marker is found, extract the step number and flush the bucket to `step_map[N]`. Trailing entries after the final marker are orphaned with a warning.

Data-quality guard: if the number of `StepRunner` markers exceeds `total_steps` from `steps.json`, the log may be from a different test run — a warning is emitted.

Format C — `raw`

No recognized structure. Returned as-is in `raw_entries` for dynamic assignment by `extract_step_logs()` in `models.py`.

4. LangGraph Node Reference

Node	Function	Input Keys Read	Output Keys Written
<code>router</code>	Context extraction, routing decision, final report assembly	<code>current_step_index</code> , <code>steps_data</code> , <code>trace_files_path</code> , <code>detected_errors</code> , <code>active_failure_map</code> , <code>step_results</code>	<code>_current_ctx</code> , <code>_route</code> , <code>_skip_reason</code> , <code>stop_execution</code> , <code>final_report</code>
<code>skip</code>	Log skip, increment counter	<code>current_step_index</code> , <code>_current_ctx</code> , <code>_skip_reason</code>	<code>current_step_index</code>
<code>vision</code>	GPT-4o vision observation	<code>_current_ctx</code> , <code>trace_files_path</code> , <code>current_step_index</code> , <code>steps_data</code>	<code>_current_vision_obs</code>
<code>log</code>	Log retrieval + GPT-4o-mini parsing	<code>_current_ctx</code> , <code>trace_files_path</code> , <code>current_step_index</code> , <code>steps_data</code>	<code>_current_log_obs</code>
<code>decider</code>	GPT-4o logic verdict	<code>_current_ctx</code> , <code>_current_vision_obs</code> , <code>_current_log_obs</code> , <code>active_failures</code>	<code>_current_decision</code>
<code>accumulate</code>	Record results, causal linking,	<code>current_step_index</code> , <code>_current_ctx</code> ,	<code>current_step_index</code> , <code>detected_errors</code> ,

Node	Function	Input Keys Read	Output Keys Written
	advance step	<code>_current_decision,</code> <code>_current_log_obs,</code> <code>detected_errors,</code> <code>active_failures,</code> <code>active_failure_map,</code> <code>step_results</code>	<code>active_failures,</code> <code>active_failure_map,</code> <code>step_results</code>

5. Pydantic Model Reference

All models are defined in `models.py`. All use Pydantic v2 semantics (`BaseModel` with `Field` defaults).

Why Pydantic for Agent I/O?

LLMs occasionally return partial JSON, wrong field types, or extra/missing keys. Pydantic:

- 1. **Validates** types at the boundary — a `confidence: float` receives a string `"0.9"` and coerces it, or raises a clear error.
- 2. **Provides defaults** — missing keys in the LLM output fall back to safe defaults (`has_logs=False`, `failure_type="none"`, etc.) rather than causing `KeyError`.
- 3. **Makes schemas explicit** — the prompt can reference the exact field names, reducing hallucination of new keys.

GraphState vs. Agent Models

`GraphState` is a `TypedDict`, not a Pydantic model. LangGraph requires `TypedDict` or dataclass for its state container. Agent outputs (`VisionObservation`, `FinalDecision`, `LogObservation`) are Pydantic models, but they are stored in `GraphState` as plain dicts (via `.model_dump()`) because `TypedDict` only stores JSON-serializable types.

6. Prompt Engineering Deep-Dive

Vision Observer Prompt Design Principles

Separation of concerns: The Observer is explicitly prohibited from making pass/fail judgments. This keeps the observation layer objective — it reports *what it sees*, not *what it means*.

Priority-ordered observation tasks:

- 1. Screen context change (navigation to new screen) — checked first
- 2. New content section (UI expansion) — checked second
- 3. Target element search — standard presence/absence check

This ordering ensures that a successful navigation (even when the trigger element disappears) is reported as a positive signal, not a missing element.

Semantic equivalence clause: Prevents false `element_missing` verdicts when an element has a language-specific or icon-based label. For example: "Çarpı" (Turkish for "cross") matches an X icon; "back" matches a

left-pointing chevron. **Exception:** numeric values and quoted strings must match character-for-character.

Strict Value OCR gate: The "VALUE NOT CONFIRMED IN POST" phrase is emitted **only** for `abstractVerification` steps with a specific data value to check. It must not appear for click/tap/navigation steps, even when those steps contain words like "verify" or "doğrula" in their instructions.

Logic Decider Prompt Design Principles

Two-phase structure: The phase separation is structural in the prompt — rules are listed first, failure history last. This exploits the LLM's tendency to commit to a decision based on the last evidence it reads before responding. By placing prior failures at the end, labeled "PHASE 2 only — Never use to decide is_failure", the model anchors its verdict on visual evidence and only uses history for root cause enrichment.

Rule priority is explicit: Rules are numbered 0, 0B, 0C, 1, 2, 3 with "Apply in order; stop at first match." This prevents the model from blending multiple rules when a specific override should apply.

Log evidence injection: The `[LOG EVIDENCE]` and `[LOG EVIDENCE RULES]` sections are conditionally appended only when logs exist. Zero bytes are added to the prompt otherwise — ensuring no performance regression for runs without logs.

Confidence calibration: The prompt defines four confidence tiers with numeric ranges and explicit qualitative descriptions. The instruction "Use the full range — do NOT anchor to 0.9 for every step" guards against a common LLM failure mode of outputting a near-constant confidence.

Log Observer Prompt Design

Cheap model justified: Log parsing is a structured extraction task (find errors, network events, set flags). It does not require multimodal capability or complex reasoning. GPT-4o-mini is sufficient and roughly 10× cheaper than GPT-4o.

Normalized input: Level casing is normalized before the API call so the model receives consistent "error" / "info" strings. This both improves accuracy (the model doesn't need to handle "ERROR" vs "error") and prevents Pydantic validation failures from the model echoing back uppercase values.

Explicit field mapping: The prompt lists the nested path to each required field (e.g., `method from request.method`, `status_code from response.status_code`) because some log formats use nested objects. The model is told the target flat structure.

7. Cascading Failure Analysis

Defined entirely in `models.py`, invoked in `graph.py`'s `accumulate_node`.

```
find_likely_cause(current_idx, current_failure_type, current_ctx,
active_failure_map)
```

Scores every prior failure in `active_failure_map` against the current failure using three factors:

Factor 1 — Element overlap (0.0–2.0):

- Current target is a substring of prior failure's instruction/target → 2.0 (strong signal)

- Prior target is a substring of current instruction/target → 1.5
- Keyword overlap (via `_normalize_keywords`) → 1.0
- No overlap → 0.0 (this factor must be > 0 for a link to be established — prevents type-correlation alone from linking unrelated failures)

Factor 2 — Failure type correlation (0.3–1.0): Uses the `_TYPE_CORRELATION` matrix. Same type = 1.0. High-propagation pairs like `action_failed` → `element_missing` = 0.8 (a failed action often leaves required elements absent). Cross-type pairs default to 0.3.

From \ To	<code>element_missing</code>	<code>content_mismatch</code>	<code>action_failed</code>	<code>system_error</code>
<code>element_missing</code>	1.0	0.6	0.8	0.3
<code>content_mismatch</code>	0.3	1.0	0.5	0.3
<code>action_failed</code>	0.8	0.8	0.9	0.5
<code>system_error</code>	0.5	0.5	0.5	0.7

Factor 3 — Temporal proximity (decay): $1.0 / \text{distance}$ where `distance` = `current_idx` - `prior_idx`. Adjacent steps score 1.0; 2 steps apart = 0.5; 3 apart = 0.33, etc.

Final score: $(\text{element_score} + \text{type_score}) \times \text{proximity_score}$

Link threshold: Score > 0.3 AND `element_score` > 0. The element overlap requirement prevents two failures from being linked purely because they have the same type and happened close together.

Returns the `step_idx` of the highest-scoring prior failure, or `None`.

`build_failure_chains(detected_errors, active_failure_map)`

Traverses the `caused_by_step` links in `detected_errors` to build a DAG, then groups failures into chains or isolates.

Algorithm:

1. Build `parent_of[child] = parent` from `caused_by_step` fields.
2. Find chain roots (failures with no parent).
3. BFS from each root to collect the full chain.
4. Format: ISOLATED FAILURE (single step) or FAILURE CHAIN N: Step X → Step Y → Step Z with root annotation.

`rank_suspicious_steps(step_results, detected_errors)`

Sorts failed steps by confidence (descending) with distance-to-chain-root as the tie-breaker.

`_resolve_root(step_idx, caused_by_map)` follows `caused_by_step` links upward (with cycle guard) to find the root of a chain and count the hop distance. A root step gets `distance_to_root=0`, its direct child gets 1, grandchild gets 2, etc.

This ranking is Section C of the final report. The intent: engineers should investigate high-confidence failures first, and prefer root failures over downstream symptoms.

`build_diagnosis_report(step_results, detected_errors, active_failure_map, total_steps)`

Assembles the final output from three sections:

Section A — Failure Chain Analysis: Output of `build_failure_chains()`.

Section C — Suspicious Step Ranking: Output of `rank_suspicious_steps()`, formatted as a ranked table with confidence, failure type, distance to root, and truncated root cause.

Section B — Summary Statistics:

- Total / analyzed / skipped / passed / failed step counts with percentages
- Confidence distribution: High (≥ 0.85), Medium (≥ 0.60), Low (< 0.60) + average
- Failure type breakdown sorted by count

8. Log Handling System

The log system has three layers, each handling a different concern.

Layer 1: `log_parser.py` — Structural Parsing (run once at startup)

Reads the full `log.json` and produces a `ParsedLogs` struct with one of:

- `step_map`: a dict mapping step index → list of log entry dicts
- `raw_entries`: the full flat list (for dynamic assignment)

This runs exactly once before `app.invoke()`. The result is installed into `graph.py`'s module-level store.

Layer 2: `extract_step_logs()` — Dynamic Assignment (in `models.py`, called per-step)

Only used when `_parsed_logs.mode == "dynamic"`. Applies keyword scoring then positional fallback to find the log entries most relevant to a specific step from the raw unstepped array.

Layer 3: `run_log_observer()` — Semantic Parsing (in `agents.py`, called per-step)

Takes the log entries (as a JSON string) and uses GPT-4o-mini to extract a structured `LogObservation`. This is the only AI call in the log pathway.

Priority Order in `log_node`

1. G411 inline logs (`_inline_console_logs` / `_inline_network_logs` in step dict)
↓ if not present
2. Preassigned `step_map` lookup (`_parsed_logs.mode == "preassigned"`)
↓ if not present
3. Dynamic keyword+positional assignment (`_parsed_logs.mode == "dynamic"`)
↓ if not present
4. Legacy per-step file (`step_{N}_logs.json` on disk)

Log Format Support Summary

Format	format_name	Mode	How step assignment works
Raw unstepped	raw	dynamic	keyword scoring + positional window at query time
Flat with step field	flat_indexed	preassigned	direct <code>step_map[step_idx]</code> lookup; 1-index shift auto-detected
Flat with StepRunner markers	flat_assertions	preassigned	markers define step boundaries; entries accumulated between markers
Array-indexed nested	nested	preassigned	array position = <code>step_idx</code> ; pipe-delimited <code>logs</code> string parsed
G411 inline	(not log_parser)	(in step dict)	<code>_inline_console_logs</code> / <code>_inline_network_logs</code> keys

9. Input Format Support

v33 Format (Native)

Standard flat array of step dicts:

```
[
  {
    "step_idx": 0,
    "action": "click",
    "element": "Login Button",
    "stepInstruction": "Click the Login button to authenticate",
    "requiresVLM": true
  },
  ...
]
```

Or wrapped in a `{"stepDetails": [...]}` envelope.

Images are named: `step_{step_idx}_prev.png` / `step_{step_idx}_post.png` (also `.jpg`, `.jpeg`).

Group 411 Format

Single JSON object with `stepDetails` array. Each item has:

- Nested `step` sub-object with only `{action, stepInstruction, condition}`
- Inline `prevScreenshot` / `currScreenshot` base64 fields (no separate image files)
- Optional `consoleLogs` / `networkLogs` plain text fields (no separate log file needed)

After `normalize_g411_steps()`, all G411 steps are indistinguishable from v33 steps except for the `_inline_*` private keys.

Action mapping applied during normalization:

- `"verify" → "abstractVerification"`
- `"type" → "click"`
- All other actions passed through unchanged

10. Error Handling & Resilience

Agent-Level Failures

Every agent call (`vision_chain.invoke`, `log_chain.invoke`, `logic_chain.invoke`) is wrapped in a try/except in the graph node. On exception:

- `vision_node` → `VisionObservation` with "System error" strings
- `log_node` → `LogObservation`(`has_logs=False`)
- `decider_node` → `FinalDecision`(`is_failure=True`, `failure_type="system_error"`)

The graph never crashes on a single step failure — it records the step as a system error and continues.

API Transport Failures

`_api_call_with_retry` handles:

- Colab socket disconnections (ENOTCONN, ECONNREFUSED)
- Rate limits and transient API errors (502, 503, timeout)
- Client re-creation before each retry (clears stale socket state)
- Exponential backoff: 2s, 4s

JSON Parse Failures

`robust_json_extract` tries two strategies before returning `{}`. An empty dict triggers the fallback path in each agent (safe defaults or system error record).

Encoding Failures

`load_project_data` tries UTF-8, UTF-8 with BOM (utf-8-sig), and CP1254 (Turkish Windows encoding) in sequence. This handles trace files generated on Turkish-locale Windows systems.

Confidence Corrections

`accumulate_node` applies post-hoc confidence adjustments that the Logic Decider cannot self-assess:

- Log parse failure detected by checking if `log_summary` starts with `"Log parsing": -0.10`
- No log data at all (`has_logs=False`): `-0.05`
- Hard floor at `0.1`

Causal Chain Gating

Low-confidence failures (confidence < 0.75) without hard log evidence cannot anchor causal chains. This prevents cascading false positives where a misread step at low confidence would incorrectly implicate all subsequent steps.

11. Performance & Configuration

Model Assignment

Role	Model	Max Tokens	Temperature	Rationale
Vision Observer	gpt-4o	500	0	Multimodal, high OCR accuracy, detail="high"
Logic Decider	gpt-4o	450	0	Complex rule-following, JSON output
Log Observer	gpt-4o-mini	600	0	Structured extraction, ~10× cheaper

temperature=0 throughout for deterministic, reproducible output — required for academic research.

Recursion Limit

config={"recursion_limit": 150} passed to app.invoke(). Covers ~70 analyzed steps with headroom (each step = ~5 graph hops: router → vision → log → decider → accumulate → router).

Memory Management

gc.collect() is called in accumulate_node after each step to reclaim memory from the large Pydantic objects and API response objects that accumulate per-step.

Colab Constraints

- nest_asyncio.apply() patches the event loop to allow async code within Colab's existing loop.
- All dependencies installed quietly (-q) to avoid cluttering notebook output.
- No GPU required — the system is fully API-based. Any Colab runtime (CPU, T4, A100) works identically.
- Google Drive is mounted lazily (only if not already mounted) to avoid double-mount errors in re-run scenarios.

Extending the System

To add a new step action type: Add a mapping in input_adapter._G411_ACTION_MAP if the new type comes from G411, and add skip conditions in router_node if the new type should bypass vision analysis.

To add a new log format: Implement a new parser function (e.g., _parse_my_format) in log_parser.py, add detection logic in _detect_format, and dispatch from parse_log_file. The rest of the pipeline consumes ParsedLogs uniformly.

To add a new failure type: Add the new string literal to FinalDecision.failure_type, update the _TYPE_CORRELATION matrix in models.py with correlation scores, and update the Decider prompt's

`failure_type` enum documentation.

To add cross-step state tracking (open issue): Add a `state_scratchpad: Dict[str, str]` field to `GraphState`. Update `accumulate_node` to extract and store key values (balances, totals) from `FinalDecision`. Inject the relevant entries as a `[STATE CONTEXT]` block in the Logic Decider prompt for `abstractVerification` steps.